LangChain - Models

By 

LangChain - model

LangChain has **multiple model wrappers**

- LLM Models – Text generation
- Chat Models and Multimodal Models
- Embedding Models
- Rerank Models

Model Type	What API Expects	LangChain Wrapper	Used For	Example Models
Chat Models	messages[] (role-based messages)	ChatOpenAI, ChatAnthropic, ChatGroq, ChatGoogleGenerativeAI, ChatHuggingFace	Conversational LLMs, multi-turn chat, RAG answer generation	GPT-4o, GPT-4o-mini, Llama 3.x (instruct/chat), Gemini 2.0 Flash, Claude 3.7
LLM Models (Text Completion)	Single text prompt	OpenAI (legacy), HuggingFacePipeline, HuggingFaceEndpoint, generic LLM wrappers	Plain text completion, one-shot generation	text-davinci-003, babbage-002, meta-llama/Llama-3 (base)
Embedding Models	Single text OR batch text → vector output	OpenAIEmbeddings, HuggingFaceEmbeddings, GoogleGenerativeAIEmbeddings, HuggingFaceInstructEmbeddings	Vectorization, retrieval, semantic search	text-embedding-3-large, all-MiniLM-L6-v2, bge-large
Rerank Models (Cross-Encoder)	Query + document (or doc list) → scores	CohereRerank, HuggingFaceCrossEncoderReranker (or HuggingFaceReranker), JinaRerank	Re-scoring retrieved docs to improve top-N selection	Cohere Rerank v3, BGE reranker, jina-reranker-v2
Multimodal Models	Mixed inputs (text + image/video/audio)	ChatOpenAI (vision/audio capable models), ChatGoogleGenerativeAI, ChatAnthropic, ChatHuggingFace	Vision QA, image grounding, multimodal reasoning	GPT-4o (vision), Gemini 2.0 Flash (multimodal), Claude (vision variants)

LLM Vs Chat Models

LLM Text Generation Models (Base Models):

Raw text prompt (no roles/messages) → text completion. Does support only openAI and Huggingface, **Gemini models** are **currently only available via the Chat wrapper** (ChatGoogleGenerativeAI) - text-generation mode is **not supported as a base LLM** in LangChain.

Examples:
- Legacy OpenAI text-davinci-003
- HF base models (Llama3 base, Falcon base, GPT-J base) via pipelines
- Custom local Transformers

Use when:
✔ Not instruction tuned
✔ Pure generation tasks

Chat Models:

Structured, role-based messages (system, user, assistant)

Examples:
- ChatOpenAI
- ChatHuggingFace
- ChatAnthropic
- ChatGoogleGenerativeAI

Use when:
✔ Agents
✔ RAG chat
✔ Multi-turn reasoning

LLM Vs Chat Models

```
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-4o")
response = model.invoke("Tell me a joke")
```

The provider's API accepts **messages**:

```
[
  {"role": "user", "content": "Hello"},
  {"role": "assistant", "content": "Hi!"}
]
```

****

```
from langchain_openai import OpenAI

model = OpenAI(model="text-davinci-003")
response = model.invoke("Write a poem")
```

The provider's API accepts **single string**:

```
{"prompt": "Write a poem"}
```

HuggingFace LLM Vs Chat Models

HuggingFace Model Types: Chat vs Text Generation

Text-Generation Models (Base LLMs)

These models take **a single text prompt** and generate continuation.

Examples

- meta-llama/Llama-3-8b
- mistralai/Mistral-7B-v0.1
- mistral-community/Mixtral-8x7B-v0.1
- EleutherAI/gpt-j-6B
- tiiuae/falcon-7b
- google/flan-t5-base
- All **base** or **pretrained** models

Chat / Instruction Models

These models are **fine-tuned for chat**, RLHF, or instruction-following.

Examples

- meta-llama/Llama-3-8b-Instruct
- meta-llama/Llama-3-8b-chat
- mistralai/Mistral-7B-Instruct
- mistralai/Mixtral-8x7B-Instruct
- Qwen/Qwen2-7B-Instruct
- google/gemma-2-9b-it
- HuggingFaceH4/zephyr-7b-alpha
- tiiuae/falcon-7b-instruct

Model name often ends with:

- Instruct
- -chat
- -it (instruction-tuned)
- -instruct

HuggingFace Local Vs Inference API

ChatHuggingFace + HuggingFaceEndpoint = HF Inference API
ChatHuggingFace + HuggingFacePipeline = HF Local deployment

```
from langchain_huggingface import HuggingFaceEndpoint,
ChatHuggingFace

llm = HuggingFaceEndpoint(

                repo_id="TinyLlama/TinyLlama-1.1B-Chat-v1.0"
                ,
                   task="text-generation",

                huggingfacehub_api_token="YOUR_TOKEN"
                )

model = ChatHuggingFace(llm=llm)

response = model.invoke("Explain RAG in two sentences.")
```

```
from transformers import pipeline
from langchain_huggingface import HuggingFacePipeline,
ChatHuggingFace

pipe = pipeline("text-generation",
model="TinyLlama/TinyLlama-1.1B-Chat-v1.0",
temperature=0.5, max_new_tokens=100)

llm = HuggingFacePipeline(pipeline=pipe)

model = ChatHuggingFace(llm=llm)

response = model.invoke("Explain RAG in two sentences.")
```

Embedding Models

❖ Produce vectors from text, docs, images, audio.

Wrappers:

- HuggingFaceEmbeddings
- OpenAIEmbeddings
- GoogleGenerativeAIEmbeddings
- OllamaEmbeddings

Use for:
✔ RAG retrieval
✔ Semantic search
✔ Chunk comparison

Ex:

```
emb = HuggingFaceEmbeddings(model_name="BAAI/bge-base-en")
```

Or

```
from langchain_openai import OpenAIEmbeddings
emb = OpenAIEmbeddings(model="text-embedding-3-large")
```

Reranker Models

❖ Rerankers improve retrieval quality by **re-scoring** the top-k retrieved documents using a **cross-encoder** model.

HuggingFace Reranker (local or API)
Popular models:

- cross-encoder/ms-marco-MiniLM-L-6-v2
- BAAI/bge-reranker-base
- jina-ai/jina-reranker-v2

Code:

```
from langchain_community.document_transformers import HuggingFaceCrossEncoderReranker

reranker = HuggingFaceCrossEncoderReranker(
    model_name="BAAI/bge-reranker-base"
)

filtered_docs = reranker.compress_documents(
    query="What is LangChain?",
    documents=docs,
)
```

Cohere Reranker model

```
from langchain_cohere import CohereRerank

reranker = CohereRerank(model="rerank-english-v3.0")

results = reranker.compress_documents(
    query="What is RAG?",
    documents=docs,  # list of Document objects
)
```

Thank you